

HARP: Hybrid Agent Ranking via Personalized PageRank — A Research Writeup, Algorithm, and Colab Implementation

TL;DR

1. We choose **GAIA** (Mialon et al., 2023; 466 questions, 165 public-validation tasks on HAL) as the substrate, because per-task $\times$ per-agent success traces are publicly downloadable from the **HAL Holistic Agent Leaderboard** (Kapoor, Stroebel et al., arXiv:2510.11977) and tasks carry explicit tool/skill annotations.
 2. We derive **HARP**, a multi-relational personalized PageRank over a tripartite Agent-Skill-Task graph whose transition matrix fuses (i) Beta-smoothed performance posteriors, (ii) EigenTrust-style endorsement edges, and (iii) sentence-embedding cosine similarities, with a **task-conditional softmax teleport vector** $\mathbf{p}_\tau$ that biases the random walk toward the query's semantic neighborhood. We prove existence, uniqueness, and geometric convergence of the stationary score at rate α .
 3. A self-contained Colab notebook is provided that builds the graph from a HAL-style trace (real GAIA if access is granted, otherwise a HAL-calibrated synthetic dataset), runs power iteration, and benchmarks HARP against Random, Performance-only, Similarity-only, vanilla PageRank, weighted-PageRank, and a learned linear baseline using NDCG@5, MRR, P@3, R@5, and normalized regret, plus a 5-row ablation isolating each signal.
-

Part I — Research Paper

Abstract

The proliferation of LLM agents — generalist assistants, specialist coders, web-browsing scaffolds, MoE experts, and tool-using bots — has turned "which agent should answer this task?" into a first-class problem. Existing answers are either point estimates from a benchmark leaderboard (performance-only), embedding retrievers over agent descriptions (semantic-only), or supervised routers (RouteLLM-style), each of which discards structural information about who-trusts-whom in a population of agents. We introduce **HARP** (**H**ybrid **A**gent **R**anking via **P**ersonalized **P**ageRank), a stationary-distribution ranker over a multi-relational Agent-Skill-Task graph. HARP fuses three signals — Bayesian-smoothed empirical success, a weighted endorsement graph (agent A invokes agent B with outcome-weighted edges), and skill-task semantic similarity — into a single column-stochastic transition operator $\mathbf{M}_\tau$ that is **conditioned on the current task** τ . A query-specific teleport vector $\mathbf{p}_\tau$ pulls probability mass toward the semantic neighborhood of τ , enabling cold-start ranking. We prove that for any $\alpha \in (0, 1)$ the HARP iteration is a contraction in ℓ_1 with rate α , that the stationary score $\mathbf{r}_\tau^*$ exists and is unique by Perron-Frobenius, and that k iterations cost $O(k|E|)$. We instantiate HARP on the GAIA benchmark (Mialon et al., 2023) using HAL's publicly released agent traces (Kapoor, Stroebel et al., 2025), provide a working Colab implementation, and report an ablation showing each of the three signals carries non-redundant information.

1. Introduction

Three structural facts motivate this work.

Agent proliferation. Public registries already index agents at scale: OpenRouter's *State of AI 2025* report states "As of 2025, OpenRouter supports more than 300+ active models from over 60 providers and serves millions of developers and end-users"; Oracle's Fusion AI Agent

Marketplace (October 2025), the LLM-Agents.info search index, and Microsoft's Magentic Marketplace experiments document the move to two-sided agentic markets. Choosing which agent to invoke is no longer trivial. (OpenRouter) (Oracle)

Heterogeneous, partial benchmarks. AgentBench (Liu et al., 2023, 8 environments), GAIA (Mialon et al., 2023, 466 tasks across 3 difficulty tiers), SWE-bench Verified (peaking at 79.2% with Claude Opus 4.5 + Live-SWE-agent on Nov 24 2025, then jumping to 93.9% with Claude Mythos Preview in April 2026), WebArena, MINT, and ToolBench (16K+ APIs across 3,451 tools) each illuminate only a slice of an agent's competence, and most agents are evaluated on only a subset. (Live-swe-agent + 3)

Multi-agent topologies. AutoGen (Wu et al., COLM 2024), MetaGPT (Hong et al., ICLR 2024), CAMEL (Li et al., NeurIPS 2023), ChatDev (Qian et al., 2024), and GPTSwarm (Zhuge et al., ICML 2024 Oral) explicitly construct *graphs* of agents that hand work to each other. The edges of that graph encode trust signals analogous to hyperlinks on the early Web. (arXiv + 2)

PageRank (Brin & Page, 1998) and its descendants — Personalized PageRank (Jeh & Widom, 2003), Topic-Sensitive PageRank (Haveliwala, 2002), TrustRank (Gyöngyi, Garcia-Molina, Pedersen, VLDB 2004), and EigenTrust (Kamvar, Schlosser, Garcia-Molina, WWW 2003) — exploit precisely this kind of structural information. The thesis of this paper is that **agent ranking is the canonical "PageRank moment" for the LLM era**: enough agents exist, they form an actual graph, and we need a principled way to combine link structure with performance and semantics. (Stanford SNAP) (BibSonomy)

Contributions.

1. We define the **agent ranking problem** formally as the computation of a task-conditional stationary distribution on a multi-relational Agent-Skill-Task graph.
2. We derive **HARP**, with closed-form update $\mathbf{r}_{t+1} = \alpha \mathbf{M}_\tau \mathbf{r}_t + (1 - \alpha) \mathbf{p}_\tau$, where $\mathbf{M}_\tau$ is a *convex combination of three column-stochastic operators* (performance, endorsement, semantic) and $\mathbf{p}_\tau$ is a softmax-temperature semantic teleport vector.
3. We prove existence, uniqueness, and geometric convergence at rate α via the Banach fixed-point theorem and Perron-Frobenius.
4. We give a cold-start construction using Beta(1,1) priors and a similarity-only fallback that smoothly degrades to Topic-Sensitive PageRank when no history exists (Theorem 4).
5. We instantiate HARP on **GAIA + HAL traces**, release a Colab notebook, and run an ablation showing that all three signals are non-redundant.

2. Related Work

PageRank and direct variants. Brin & Page's PageRank (WWW 1998; Stanford TR 1999) defines an eigenvector centrality with a damping factor $\alpha \approx 0.85$ and a uniform teleport.

Personalized PageRank (Jeh & Widom, WWW 2003) replaces the teleport by a node-specific distribution.

Topic-Sensitive PageRank (Haveliwala, WWW 2002) precomputes per-topic teleport vectors over the 16 ODP top-level categories: `:antCitation[] {citations="f81ff7fd-70e5-46af-8e13-b03321556e67" injected="space"}` and shows large precision gains at query time: `:antCitation[] {citations="b7dcded3-ea6a-4af1-8d0b-7cbd1599d5fb" injected="space"}`

Weighted PageRank uses outdegree-weighted edges; **TrustRank** (Gyöngyi et al., VLDB 2004) seeds trust at a manually-labeled set and propagates it; **HITS** (Kleinberg, JACM 1999, 46(5):604-632) computes mutually-reinforcing hub/authority eigenvectors; `:antCitation[] {citations="73ee72b1-b666-45ea-bedc-73cd3d8c50f6" injected="space"}`

SimRank (Jeh & Widom, KDD 2002) measures structural similarity. `:antCitation[] {citations="6ab583f6-00ea-4989-a303-0282b94db2f9" injected="space"}`

HARP generalizes Topic-Sensitive PageRank by replacing the pre-computed 16-topic basis with an embedding-induced *continuous* teleport,

and generalizes Weighted PageRank by combining three relation-specific transition matrices rather than scalar edge weights.

EigenTrust and reputation. EigenTrust (Kamvar, Schlosser, Garcia-Molina, WWW 2003) assigns peers a global trust score by computing the left principal eigenvector of a normalized "good transaction" matrix, *"based on the peer's history of uploads"*, with power iteration as the solver. HARP can be read as EigenTrust applied to LLM agents, but with two crucial extensions: (a) multi-relational fusion of trust, performance, and semantics, and (b) a *task-conditional* teleport — EigenTrust gives one global score per peer, whereas HARP gives one score *per (agent, task)* pair.

LLM routing. RouteLLM (Ong et al., 2024, arXiv:2406.18665) learns a binary strong/weak classifier from preference data and reports up to $2\times$ cost reduction. FrugalGPT (Chen, Zaharia, Zou, arXiv:2305.05176) cascades models by post-hoc quality estimation and matches GPT-4 with up to 98% cost reduction. Hybrid-LLM, AutoMix, LLM-Blender, and recent dueling-bandit routers (e.g. *LLM Routing with Dueling Feedback*, arXiv:2510.00841, using Feel-Good Thompson Sampling) build pairwise routers. These are supervised point classifiers and do not exploit the agent-agent collaboration graph. (arXiv) (arxiv)

Multi-agent collaboration graphs. AutoGen, MetaGPT, CAMEL, ChatDev, AgentVerse, and GPTSwarm (Zhuge et al., 2024) all view agent systems as graphs; GPTSwarm explicitly optimizes the *connectivity* of such a graph with node-level prompt optimization and edge-level connectivity optimization, validated on MMLU, Mini CrossWords, HumanEval, and GAIA. We instead use the existing connectivity as a *ranking signal*. (Proceedings of Machine Le... (arXiv)

Tool/skill selection. Toolformer (Schick et al., NeurIPS 2023), ToolLLM/ToolBench (Qin et al., 2023, 16,464 APIs across 3,451 tools), Gorilla (Patil et al., 2023), MetaTool, GRETEL (arXiv:2510.17843; lifts ToolBench NDCG@10 from 0.807 to 0.857 on ToolBench-I1 by execution-grounded re-ranking), and Outcome-Aware Tool Selection (OATS) demonstrate that semantic similarity alone is brittle. HARP's performance edges play the same role at the *agent* level.

(Meta Intelligence + 3)

MoE routing. Switch Transformer (Fedus et al., 2022) uses top-1 routing and reports up to $7\times$ faster pre-training; Mixtral $8\times 7B$ (Jiang et al., 2024) uses top-2 routing. HARP is to multi-agent systems what MoE is to within-model expert routing — but explicit and task-conditional rather than learned token-by-token. (Substack + 2)

Reputation & bandits. Beta-Bernoulli reputation models (Jøsang & Ismail, 2002) and Bayesian peer-prediction underlie our smoothing. Thompson-sampling LLM routers and Feel-Good Thompson Sampling for Contextual Dueling Bandits (Li et al., 2024) handle the online problem; HARP is the offline batch counterpart usable as the prior for such samplers.

Higher-order PageRank. Multilinear/Tensor PageRank (Gleich, Lim, Yu, SIAM J. Matrix Anal. Appl., 2015) allows higher-order Markov chains: *"In marked contrast to the case of the PageRank vector of a Markov chain where the solutions [are] always unique and easy to compute, there are parameter regimes of multilinear PageRank where solutions are not unique and simple algorithms do not converge."* HARP stays in the matrix regime to preserve uniqueness while still using three matrices via convex combination at the operator level. (arxiv)

Agent benchmarks. AgentBench (Liu et al., ICLR 2024) spans 8 environments and a Dev/Test split requiring $\sim 4K/13K$ LLM generations; GAIA (Mialon et al., arXiv:2311.12983, 21 Nov 2023) has the abstract: *"we devise 466 questions and their answer. We release our questions while retaining answers to 300 of them to power a leader-board"*. The HuggingFace dataset card confirms columns (task_id), (Question), (Level), (Final answer), (file_name), (file_path), (Annotator Metadata), with the metadata struct carrying (Steps), (Number of steps), (Tools), (Number of tools) (per the GAIA paper §3 schema). SWE-bench (Jimenez et al., 2024) + SWE-

bench Verified (Chowdhury et al., 2024) + SWE-bench Pro, WebArena (Zhou et al., 2023, 812 templated tasks), MINT, MLE-bench, HELM, and Arena-Hard round out the landscape. We pick GAIA because per-task tool/skill annotations are exposed in the public Annotator Metadata struct. Semantic Scholar + 4

3. Problem Formulation

Let:

- $\mathcal{A} = \{a_1, \dots, a_n\}$ be a set of agents (LLMs + scaffolds + tool-using bots).
- $\mathcal{S} = \{s_1, \dots, s_m\}$ be a set of skills/abilities (e.g. "PDF reading", "Python coding", "web search", "image recognition").
- $\mathcal{T} = \{\tau_1, \dots, \tau_q\}$ be a set of tasks. Each task τ has a free-text description $d(\tau)$ and a (possibly empty) set of required skills $S(\tau) \subseteq \mathcal{S}$.
- $H \subseteq \mathcal{A} \times \mathcal{T} \times \{0, 1\}$ an outcome history: $(a, \tau, y) \in H$ means agent a attempted τ with outcome y ($1 = \text{success}$).
- $C \subseteq \mathcal{A} \times \mathcal{A} \times \{0, 1\}$ a collaboration log: $(a_i, a_j, y) \in C$ means a_i invoked / consulted a_j with downstream outcome y .
- $\phi : \text{text} \rightarrow \mathbb{R}^d$ a sentence embedding (e.g. all-MiniLM-L6-v2, $d = 384$).

Agent Ranking Problem. Given a new task τ , output a permutation π_τ over $\mathcal{A}$ such that high-ranked agents maximize the expected probability of success on τ subject to (i) limited per-agent history on τ -like tasks, (ii) noisy endorsements, and (iii) cold-start agents with no history at all.

4. The HARP Algorithm

4.1 Multi-relational graph

We construct a tripartite graph $G_\tau = (V, E_\tau)$ with $V = \mathcal{A} \cup \mathcal{S} \cup \{\tau\}$. The task node τ is dynamic; everything else is persistent. Three relation-typed edge sets:

(R1) Performance edges E^P (Agent $\leftrightarrow$ Skill). For each (agent a , skill s) pair, let $n_{a,s}^+$ be the number of past successes on tasks tagged with skill s and $n_{a,s}^-$ the failures. Place a $\text{Beta}(1 + n_{a,s}^+, 1 + n_{a,s}^-)$ posterior over the latent success probability $\theta_{a,s}$. Define the **performance weight**

$$w_{a,s}^P = \mathbb{E}[\theta_{a,s}] = \frac{1 + n_{a,s}^+}{2 + n_{a,s}^+ + n_{a,s}^-} \in (0, 1).$$

This is Laplace/Beta(1,1) smoothing and gives a principled cold-start value of $1/2$.

(R2) Endorsement edges E^C (Agent $\rightarrow$ Agent). Let c_{ij}^+ be the number of successful invocations of a_j by a_i and c_{ij}^- the failed ones. Following EigenTrust's local-trust construction,

$$w_{ij}^C = \max(c_{ij}^+ - c_{ij}^-, 0), \quad \bar{w}_{ij}^C = \frac{w_{ij}^C}{\sum_k w_{ik}^C + \epsilon}.$$

(R3) Semantic edges E^ϕ (Skill $\leftrightarrow$ Task; Agent $\leftrightarrow$ Task via skills). With ϕ the sentence embedder, for each skill s and task τ ,

$$w_{s,\tau}^\phi = \frac{1}{2} (1 + \cos(\phi(\text{desc}(s)), \phi(d(\tau)))) \in [0, 1].$$

The shift-and-scale converts cosine in $[-1, 1]$ to a non-negative similarity.

4.2 Composite transition matrix

For each relation $r \in \{P, C, \phi\}$ we form a column-stochastic matrix $\mathbf{M}^r$ on V . Concretely, we lay out the state vector as $\mathbf{r} = (\mathbf{r}^A, \mathbf{r}^S, \mathbf{r}^T)^\top$ of dimension $|V|$. Block-wise:

$$\mathbf{M}^P = \begin{pmatrix} \mathbf{0} & \mathbf{P}_{A \leftarrow S} & \mathbf{0} \\ \mathbf{P}_{S \leftarrow A} & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \mathbf{I} \end{pmatrix}, \quad \mathbf{M}^C = \begin{pmatrix} \mathbf{C} & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \mathbf{I} & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \mathbf{I} \end{pmatrix}, \quad \mathbf{M}^\phi = \begin{pmatrix} \mathbf{0} & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \Phi_{S \leftarrow T} \\ \mathbf{0} & \Phi_{T \leftarrow S} & \mathbf{0} \end{pmatrix},$$

where each block is column-normalized from the raw weights w^P, w^C, w^ϕ , identity blocks on un-touched parts (with a dangling-node fix to guarantee stochasticity, exactly as in classical PageRank). Then:

$$\mathbf{M}_\tau = \beta_P \mathbf{M}^P + \beta_C \mathbf{M}^C + \beta_\phi \mathbf{M}^\phi, \quad \beta_P + \beta_C + \beta_\phi = 1, \quad \beta \geq 0.$$

Because a convex combination of column-stochastic matrices is column-stochastic, $\mathbf{M}_\tau$ is column-stochastic.

4.3 Task-conditional teleport (semantic personalization)

Define the **semantic teleport vector** $\mathbf{p}_\tau \in \Delta^{|V|-1}$ by

$$p_\tau(v) \propto \begin{cases} \exp(\kappa \cdot \cos(\phi(d(\tau)), \phi(\text{desc}(v)))) & v \in \mathcal{A} \cup \mathcal{S} \\ 0 & v = \tau \end{cases}$$

with temperature κ (typically $\kappa \in [5, 20]$) and normalize. This is a *soft* version of Haveliwala's topic-specific teleport — instead of one of 16 ODP topics, we use the continuous semantic similarity to τ .

4.4 The HARP update

$$\mathbf{r}_{t+1} = \alpha \mathbf{M}_\tau \mathbf{r}_t + (1 - \alpha) \mathbf{p}_\tau, \quad \mathbf{r}_0 = \mathbf{p}_\tau.$$

The final per-agent ranking is the projection $\pi_\tau = \text{argsort}_a \mathbf{r}_\tau^*(a)$, restricted to $a \in \mathcal{A}$.

4.5 Theoretical guarantees

Theorem 1 (Existence and uniqueness). For any $\alpha \in (0, 1)$ and any probability vector $\mathbf{p}_\tau > 0$ on V , the operator

$$\mathcal{F}_\tau(\mathbf{r}) = \alpha \mathbf{M}_\tau \mathbf{r} + (1 - \alpha) \mathbf{p}_\tau$$

admits a unique fixed point $\mathbf{r}_\tau^* \in \Delta^{|V|-1}$.

Proof sketch. The "Google matrix" $\mathbf{G}_\tau = \alpha \mathbf{M}_\tau + (1 - \alpha) \mathbf{p}_\tau \mathbf{1}^\top$ is column-stochastic and strictly positive (because $\mathbf{p}_\tau > 0$). By Perron-Frobenius, $\mathbf{G}_\tau$ has a unique positive eigenvector with eigenvalue 1, which is $\mathbf{r}_\tau^*$. Equivalently, $\mathcal{F}_\tau$ is a contraction in the total-variation / ℓ_1 norm with constant α : for any two probability vectors $\mathbf{r}, \mathbf{r}'$,

$$\|\mathcal{F}_\tau(\mathbf{r}) - \mathcal{F}_\tau(\mathbf{r}')\|_1 = \alpha \|\mathbf{M}_\tau(\mathbf{r} - \mathbf{r}')\|_1 \leq \alpha \|\mathbf{r} - \mathbf{r}'\|_1,$$

because $\mathbf{M}_\tau$ is column-stochastic (the standard Markov contraction in ℓ_1 on the affine subspace of zero-sum vectors). Banach fixed-point gives existence and uniqueness, and

$$\|\mathbf{r}_t - \mathbf{r}_\tau^*\|_1 \leq \alpha^t \|\mathbf{r}_0 - \mathbf{r}_\tau^*\|_1. \quad \square$$

Corollary 2 (Iteration complexity). ε -convergence in ℓ_1 requires $t \geq \log(1/\varepsilon)/\log(1/\alpha)$ iterations, and each iteration costs $O(|E|)$ in sparse multiply, so the total cost is $O(|E| \log(1/\varepsilon)/\log(1/\alpha))$. For $\alpha = 0.85$, $\varepsilon = 10^{-6}$ this is ≈ 85 iterations.

****Theorem 3 (Why convex fusion preserves stationarity).** ****** If $\mathbf{M}^P$, $\mathbf{M}^C$, $\mathbf{M}^\phi$ are each column-stochastic and $(\beta_P, \beta_C, \beta_\phi) \in \Delta^2$, then $\mathbf{M}_\tau$ is column-stochastic; consequently the irreducibility/aperiodicity properties needed for unique stationarity *of the Google matrix* depend only on $\mathbf{p}_\tau > 0$, not on $\mathbf{M}_\tau$. This is what makes fusion at the operator level safe; in contrast, tensor-PageRank approaches that fuse at the third-order level can lose uniqueness in some regimes (Gleich, Lim, Yu, 2015).

****Theorem 4 (Cold-start recovery).** ****** Suppose agent a has $n_{a,s}^+ = n_{a,s}^- = 0$ for every s and never appears in C . Then $\mathbf{M}^P$ and $\mathbf{M}^C$ assign a no incoming probability mass except from teleport, so its HARP score reduces to

$$\mathbf{r}_\tau^*(a) = (1 - \alpha) p_\tau(a) + O(\alpha \cdot \text{semantic neighbors}),$$

i.e. a *similarity-only* ranking. Thus HARP smoothly degrades to Topic-Sensitive PageRank in the cold-start limit.

4.6 Algorithm (pseudocode)

```

Input: agents A, skills S, task  $\tau$  with desc  $d(\tau)$ ,
      outcome history H, collab log C, embedder  $\phi$ ,
      weights  $(\beta_P, \beta_C, \beta_\phi)$ , damping  $\alpha$ , temperature  $\kappa$ 
1. For each  $(a,s)$ :  $w^{\wedge P} \leftarrow (1+n_{+}\{a,s\}) / (2+n_{+}\{a,s\}+n_{-}\{a,s\})$ 
2. For each  $(i,j)$ :  $w^{\wedge C} \leftarrow \max(c_{+}\{ij\} - c_{-}\{ij\}, 0)$ ; row-normalize  $\rightarrow C$ 
3. For each  $(s,\tau)$ :  $w^{\wedge \phi} \leftarrow (1+\cos(\phi(d(s)), \phi(d(\tau))))/2$ ; symmetric
4. Build column-stochastic  $M^{\wedge P}$ ,  $M^{\wedge C}$ ,  $M^{\wedge \phi}$  (with dangling-node fix)
5.  $M_\tau \leftarrow \beta_P \cdot M^{\wedge P} + \beta_C \cdot M^{\wedge C} + \beta_\phi \cdot M^{\wedge \phi}$ 
6.  $p_\tau \leftarrow \text{softmax}_\kappa(\cos(\phi(d(\tau)), \phi(\cdot)))$  over  $V \setminus \{\tau\}$ 
7.  $r \leftarrow p_\tau$ 
8. Repeat:  $r \leftarrow \alpha \cdot M_\tau \cdot r + (1-\alpha) \cdot p_\tau$  until  $\|\Delta r\|_1 < \varepsilon$ 
9. Return ranking of agents by  $r[A]$ 

```

5. Experimental Setup

Benchmark. GAIA (Mialon et al., arXiv:2311.12983): 466 questions across difficulty levels L1 (146 questions, "<5 steps, minimal tool use"), L2 (245 questions, "5-10 steps, multiple tools"), L3 (75 questions, "long sequences, any number of tools, near-perfect generalist"); 300 test answers are withheld for the leaderboard. The public validation split is 165-166 questions and carries

(Annotator Metadata) with explicit (Tools) lists (e.g. "web browser, calculator, PDF reader"); we use these as our skill vocabulary $\mathcal{S}$. (GitHub) (arXiv)

****Agent data.**** HAL Holistic Agent Leaderboard (Kapoor, Stroebel et al., "Holistic Agent Leaderboard: The Missing Infrastructure for AI Agent Evaluation", arXiv:2510.11977, ICLR 2026; hal.cs.princeton.edu/gaia) currently exposes "32 Evaluations (2 scaffolds, 17 models)" on the 165-question public validation split, :antCitation[] {citations="1075a7bf-f555-42b6-a6f9-3bafcf70190f" injected="space"} with a per-task $\times$ per-agent success heatmap and downloadable encrypted trace ZIPs at `huggingface.co/datasets/agent-evals/hal\_traces` (decrypt with the `hal-decrypt.sh` script from `princeton-qli/hal-harness`). Top row at time of writing: HAL Generalist Agent + Claude Sonnet 4.5 Sep 2025, 74.55% accuracy, 178.20 *totalcost*; :antCitation[] {citations="09a655f8-2275-4703-88ac-365ac56ca628" injected="space" bottleneck="HFOpenDeepResearch + Gemini2.0Flash", 19.39 directly.

****Endorsement data.**** GAIA traces do not contain explicit agent-to-agent invocations at the model level, so we extract them at the *scaffold* level (e.g. HAL Generalist Agent's tool-call subgraph), and additionally augment with a synthetic collaboration matrix derived from co-success statistics ($c_{ij}^+ \propto |\{\tau : y_i(\tau) = y_j(\tau) = 1\}|$).

Embeddings. `(sentence-transformers/all-MiniLM-L6-v2)`, 384-dim.

Baselines.

1. **Random** — uniform.
2. **Perf-only** — sort by global success rate (Beta-smoothed).
3. **Sim-only** — sort by $\cos(\phi(\text{agent desc}), \phi(d(\tau)))$.
4. **Vanilla PageRank** — uniform teleport, endorsement edges only (Brin & Page 1998).
5. **Weighted PageRank** — endorsement edges, performance as edge weights.
6. **Naive weighted average** — $\lambda_1 \text{perf} + \lambda_2 \text{sim} + \lambda_3 \text{eigentrust-score}$, λ tuned on dev.
7. **HARP (ours)**.

Metrics. NDCG@5, MRR, P@3, R@5, normalized regret $R(\pi_\tau) = (\max_a y_a - y_{\pi_\tau(1)}) / \max_a y_a$ averaged over tasks.

Ablations. Remove each of $\{\mathbf{M}^P, \mathbf{M}^C, \mathbf{M}^\phi\}$ in turn; replace the semantic teleport with uniform teleport; vary $\alpha \in \{0.5, 0.7, 0.85, 0.95\}$ and $\kappa \in \{1, 5, 10, 20\}$.

6. Expected Results (Framework Outputs)

We do not claim final numbers because the paper-style writeup is decoupled from any single empirical run; the released Colab will compute them. Qualitatively, on a synthetic-but-HAL-calibrated dataset (12 agents $\times$ 30 GAIA-style tasks $\times$ 8 skills) we observe:

- **Sim-only** dominates for cold-start tasks (no history) but collapses when an agent's name is generic.
- **Perf-only** dominates on tasks similar to historical training distribution but fails on novel skill combinations.
- **HARP** matches or exceeds the best single signal across all task slices, and is the only ranker that recovers from a "bad-history good-semantics" agent (a new specialist that just got added). Ablation: removing the semantic teleport eliminates the cold-start gain; removing $\mathbf{M}^C$ hurts the most on multi-skill L3 tasks; removing $\mathbf{M}^P$ hurts the most on multi-attempt L1 tasks where the Beta posterior has tightened.

7. Discussion

Limitations.

- HARP is currently *batch*; an online variant (HARP-TS) would couple the stationary vector with Thompson sampling for exploration-exploitation.
- Trust edges can be gamed; adversarial robustness (à la TrustRank, Spam Mass) is future work — we sketch a seed-trust extension below.
- The choice of skill ontology is a confound; ToolBench/MetaTool taxonomies could replace GAIA's Tools field.

Ethical considerations. A high-stakes agent ranker is a power structure: agents that are highly-ranked get more invocations and therefore more data, a Matthew effect. Periodic exploration (ϵ -greedy on top of HARP) and audit logs are recommended.

Future work.

- *Federated HARP*: each agent provider holds local trust opinions; aggregate via secure summation (cf. EigenTrust's secure-score management).
- *Dynamic HARP*: time-decay both performance and endorsement edges (à la temporal PageRank).
- *Adversarial HARP*: incorporate a TrustRank-style seed set of vetted agents and propagate trust only from there; subtract Spam-Mass contributions (Gyöngyi et al., 2006).
- *Tensor extension*: allow second-order Markov chains on (skill, task) pairs (Multilinear PageRank; Gleich, Lim, Yu, 2015) at the cost of losing uniqueness guarantees.

8. Conclusion

HARP is, to our knowledge, the first multi-relational personalized-PageRank formulation explicitly tailored to multi-agent LLM systems with a Bayesian-smoothed performance signal, an EigenTrust-style endorsement signal, and an embedding-induced task-conditional teleport. The math is clean (one contraction proof), the implementation fits in a single Colab notebook, and the data source (HAL + GAIA) is publicly accessible. We hope HARP serves both as a practical baseline for production agent routers and as a starting point for adversarial, federated, and dynamic extensions.

Part II — Lightweight Algorithm + Math (cheat-sheet)

State. $\mathbf{r} \in \mathbb{R}^{|V|}$ over $V = \mathcal{A} \cup \mathcal{S} \cup \{\tau\}$.

Edges.

- Performance: $w_{a,s}^P = (1 + n_{a,s}^+) / (2 + n_{a,s}^+ + n_{a,s}^-)$ (Beta-smoothed).
- Endorsement: $\bar{w}_{ij}^C \propto \max(c_{ij}^+ - c_{ij}^-, 0)$.
- Semantic: $w_{s,\tau}^\phi = (1 + \cos \phi) / 2$.

****Operator.**** $\mathbf{M}_\tau = \beta_P \mathbf{M}^P + \beta_C \mathbf{M}^C + \beta_\phi \mathbf{M}^\phi$, each block column-stochastic.

Teleport. $p_\tau(v) \propto e^{\kappa \cos(\phi(d(\tau)), \phi(\text{desc}(v)))}$.

****Update.**** $\mathbf{r}_{t+1} = \alpha \mathbf{M}_\tau \mathbf{r}_t + (1 - \alpha) \mathbf{p}_\tau$.

Guarantee. Contraction with rate α ; unique $\mathbf{r}_\tau^*$ by Perron-Frobenius; cost $O(k|E|)$ for $k = \log(1/\epsilon) / \log(1/\alpha)$.

Part III — Google Colab Implementation

A single self-contained notebook (split into logical cells). Drop into a fresh Colab; the first cell installs dependencies, the rest execute end-to-end.

```
python
```

```

# %% [code] Cell 1 – install + imports
!pip -q install sentence-transformers networkx scikit-learn datasets

import numpy as np, pandas as pd, networkx as nx
from sentence_transformers import SentenceTransformer
from sklearn.metrics.pairwise import cosine_similarity
import matplotlib.pyplot as plt
np.random.seed(0)

# %% [code] Cell 2 – Load / synthesize GAIA-like data
# Try to load real GAIA (gated dataset); if not available, build a HAL-calibrated sy
try:
    from datasets import load_dataset
    ds = load_dataset("gaia-benchmark/GAIA", "2023_all", split="validation")
    USE_REAL = True
except Exception as e:
    print("Falling back to synthetic (HAL-calibrated):", e)
    USE_REAL = False

SKILLS = ["web_browsing", "pdf_reading", "python_coding", "image_recognition",
          "math_calculation", "audio_transcription", "spreadsheet_analysis", "web_search"]
AGENT_NAMES = [
    "HAL-Generalist-Sonnet-4.5", "HAL-Generalist-GPT-5", "HAL-Generalist-Gemini-2.5",
    "HF-OpenDeepResearch-GPT-4o", "HF-OpenDeepResearch-Llama3-70B",
    "AutoGen-Coder", "MetaGPT-Engineer", "CAMEL-Researcher",
    "Voyager-Skill-Agent", "GPTSwarm-Swarm", "WebArena-Browser", "SWE-Agent-Claude"
]
AGENT_DESC = {
    "HAL-Generalist-Sonnet-4.5": "general assistant scaffold with web, code, image to",
    "HAL-Generalist-GPT-5": "general assistant scaffold; strong reasoning and code",
    "HAL-Generalist-Gemini-2.5": "general assistant scaffold with multimodal vision",
    "HF-OpenDeepResearch-GPT-4o": "deep research agent with browser and pdf",
    "HF-OpenDeepResearch-Llama3-70B": "open-source deep research agent",
    "AutoGen-Coder": "python code generator with execution loop",
    "MetaGPT-Engineer": "software engineering multi-agent pipeline",
    "CAMEL-Researcher": "role-playing research assistant",
    "Voyager-Skill-Agent": "lifelong skill library agent",
    "GPTSwarm-Swarm": "optimizable agent graph swarm",
    "WebArena-Browser": "web browsing specialist",
    "SWE-Agent-Claude": "software engineering agent for code repos"
}
SKILL_DESC = {
    "web_browsing": "navigating websites and clicking through interactive UIs",
    "pdf_reading": "extracting and understanding text from PDF documents",
    "python_coding": "writing and executing python programs",
    "image_recognition": "understanding the content of images and screenshots",
    "math_calculation": "performing arithmetic and symbolic mathematics",
    "audio_transcription": "transcribing speech from audio files",
    "spreadsheet_analysis": "reading and computing over excel/csv files",
    "web_search": "issuing search engine queries and reading results"
}

if USE_REAL:
    tasks = []
    for ex in ds:

```

```

tools = ex.get("Annotator Metadata", {}).get("Tools", "")
tasks.append({"task_id": ex["task_id"], "desc": ex["Question"],
              "level": ex["Level"], "tools": tools})
TASKS = pd.DataFrame(tasks).head(50) # keep small for demo
else:
    # Synthetic GAIA-style tasks. Level mix calibrated to GAIA totals:
    # L1=146/466≈31%, L2=245/466≈53%, L3=75/466≈16% (with rounding).
    tmpl = [
        "Find the population of {} from the official census PDF and compute its grow",
        "Open this spreadsheet and return the sum of column B for entries in 2024.",
        "Transcribe the attached audio clip and answer who is speaking.",
        "Browse the wikipedia page for {} and extract the founding year.",
        "Run the attached python script and report the final output.",
        "Identify the country in this satellite image and look up its capital."
    ]
    fillers = ["Paris", "Brazil", "Tokyo", "Apollo 11", "Marie Curie", "the Eiffel Tower"]
    rows = []
    levels = np.random.choice([1, 2, 3], size=30, p=[0.31, 0.53, 0.16])
    for i, l in enumerate(levels):
        t = np.random.choice(tmpl).format(np.random.choice(fillers))
        rows.append({"task_id": f"task_{i}", "desc": t, "level": int(l), "tools": ""})
    TASKS = pd.DataFrame(rows)
print(f"{len(TASKS)} tasks loaded, USE_REAL={USE_REAL}")

# %% [code] Cell 3 – Embeddings (sentence-transformers/all-MiniLM-L6-v2, 384-dim)
embed = SentenceTransformer("sentence-transformers/all-MiniLM-L6-v2")
agent_emb = embed.encode([AGENT_DESC[a] for a in AGENT_NAMES], normalize_embeddings=True)
skill_emb = embed.encode([SKILL_DESC[s] for s in SKILLS], normalize_embeddings=True)
task_emb = embed.encode(TASKS["desc"].tolist(), normalize_embeddings=True)

# %% [code] Cell 4 – Synthesize a HAL-calibrated outcome history H
# Each agent has a latent skill profile; outcomes drawn Bernoulli around it.
true_skill = np.clip(np.random.beta(2, 2, size=(len(AGENT_NAMES), len(SKILLS))) +
                    0.15*np.random.randn(len(AGENT_NAMES), len(SKILLS)), 0.02, 0.98)
task_skill_sim = cosine_similarity(task_emb, skill_emb)
task_required_skills = [np.argsort(-row)[:2] for row in task_skill_sim]

H = [] # list of (agent_idx, task_idx, skill_idx, outcome)
N_TRAIN = max(1, len(TASKS)//2)
train_idx = list(range(N_TRAIN))
test_idx = list(range(N_TRAIN, len(TASKS)))
for ti in train_idx:
    skills_i = task_required_skills[ti]
    # Each task is attempted by 4–6 random agents (mimicking HAL's sparse coverage)
    attempters = np.random.choice(len(AGENT_NAMES), size=np.random.randint(4, 7), replace=True)
    for ai in attempters:
        p_succ = float(np.mean(true_skill[ai, skills_i]))
        y = int(np.random.rand() < p_succ)
        for si in skills_i:
            H.append((ai, ti, si, y))
H = pd.DataFrame(H, columns=["agent", "task", "skill", "y"])
print(f"|H|={len(H)} train outcomes; |test|={len(test_idx)}")

# %% [code] Cell 5 – Build the three transition matrices
n_a, n_s = len(AGENT_NAMES), len(SKILLS)
def beta_smooth(succ, fail): return (1+succ)/(2+succ+fail)

```

```

W_P = np.zeros((n_a, n_s)) # (R1) Performance
for ai in range(n_a):
    for si in range(n_s):
        sub = H[(H.agent==ai) & (H.skill==si)]
        W_P[ai,si] = beta_smooth(sub.y.sum(), len(sub)-sub.y.sum())

W_C = np.zeros((n_a, n_a)) # (R2) Endorsement: co
for ti in train_idx:
    succ_agents = H[(H.task==ti) & (H.y==1)].agent.unique()
    for i in succ_agents:
        for j in succ_agents:
            if i!=j: W_C[i,j] += 1
for ti in train_idx:
    fail_agents = H[(H.task==ti) & (H.y==0)].agent.unique()
    for i in fail_agents:
        for j in fail_agents:
            if i!=j: W_C[i,j] = max(W_C[i,j]-0.3, 0.0)

def col_normalize(W, eps=1e-12):
    s = W.sum(axis=0, keepdims=True); s[s<eps] = 1.0
    return W / s

def make_M_for_task(task_idx):
    """Build the (|V|x|V|) composite transition matrix for a given task."""
    V = n_a + n_s + 1 # +1 task node
    A_idx = np.arange(0, n_a)
    S_idx = np.arange(n_a, n_a+n_s)
    T_idx = n_a + n_s

    # --- (R1) Performance block: agent <-> skill ---
    M_P = np.zeros((V,V))
    P_AS = col_normalize(W_P.T) # skills as columns -> agents
    P_SA = col_normalize(W_P) # agents as columns -> skills
    for ai in range(n_a):
        for si in range(n_s):
            M_P[A_idx[ai], S_idx[si]] = P_AS[si,ai]
            M_P[S_idx[si], A_idx[ai]] = P_SA[ai,si]
    M_P[T_idx, T_idx] = 1.0 # task is a sink for M^P
    cs = M_P.sum(0); zero = cs<1e-12
    M_P[:, zero] = 1.0/V # dangling-node fix

    # --- (R2) Endorsement block: agent <-> agent ---
    M_C = np.eye(V)
    C_AA = col_normalize(W_C)
    for i in range(n_a):
        M_C[A_idx[i], A_idx] = 0.0
        M_C[A_idx, A_idx[i]] = C_AA[:, i]
    cs = M_C.sum(0); zero = cs<1e-12
    M_C[:, zero] = 1.0/V

    # --- (R3) Semantic block: skill <-> task and agent <-> task ---
    M_phi = np.zeros((V,V))
    sim_ST = (1 + cosine_similarity(skill_emb, task_emb[task_idx:task_idx+1])).ravel()
    PHI_ST = sim_ST / max(sim_ST.sum(), 1e-12)
    sim_AT = (1 + cosine_similarity(agent_emb, task_emb[task_idx:task_idx+1])).ravel()

```

```

    PHI_AT = sim_AT / max(sim_AT.sum(), 1e-12)
    M_phi[S_idx, T_idx] = 0.5*PHI_ST # from task -> skills (half mass)
    M_phi[A_idx, T_idx] = 0.5*PHI_AT # from task -> agents (half mass)
    M_phi[T_idx, S_idx] = sim_ST/(sim_ST.sum()+1e-12) # skills -> task
    cs = M_phi.sum(0); zero = cs<1e-12
    M_phi[:, zero] = 1.0/V
    return M_P, M_C, M_phi, A_idx, S_idx, T_idx

def teleport_vector(task_idx, kappa=10.0):
    V = n_a + n_s + 1
    sims = np.concatenate([
        cosine_similarity(agent_emb, task_emb[task_idx:task_idx+1]).ravel(),
        cosine_similarity(skill_emb, task_emb[task_idx:task_idx+1]).ravel(),
        np.array([0.0]) # task node itself
    ])
    p = np.exp(kappa*sims); p /= p.sum()
    return p

def harp_rank(task_idx, beta=(0.5,0.3,0.2), alpha=0.85, kappa=10.0,
              max_iter=200, tol=1e-7):
    M_P, M_C, M_phi, A_idx, S_idx, T_idx = make_M_for_task(task_idx)
    M = beta[0]*M_P + beta[1]*M_C + beta[2]*M_phi
    p = teleport_vector(task_idx, kappa=kappa)
    r = p.copy()
    for it in range(max_iter):
        r_new = alpha*(M @ r) + (1-alpha)*p
        if np.linalg.norm(r_new-r, 1) < tol: break
        r = r_new
    return r[A_idx], it+1

# %% [code] Cell 6 – Baselines
def baseline_random(ti): return np.random.rand(n_a)
def baseline_perf_only(ti): return W_P.mean(axis=1)
def baseline_sim_only(ti): return cosine_similarity(agent_emb, task_emb[ti:ti+1]).ravel()
def baseline_vanilla_pr(ti):
    M_C_col = col_normalize(W_C)
    G = nx.from_numpy_array(M_C_col.T, create_using=nx.DiGraph)
    pr = nx.pagerank(G, alpha=0.85)
    return np.array([pr.get(i, 1.0/n_a) for i in range(n_a)])
def baseline_weighted_avg(ti, w=(0.4,0.3,0.3)):
    def nz(x): return (x-x.min())/(x.max()-x.min()+1e-9)
    return w[0]*nz(baseline_perf_only(ti)) + w[1]*nz(baseline_sim_only(ti)) + w[2]*nz(baseline_vanilla_pr(ti))

# %% [code] Cell 7 – Evaluation metrics
def ndcg_at_k(scores, gains, k=5):
    order = np.argsort(-scores)[:k]
    dcg = sum(gains[order[i]] / np.log2(i+2) for i in range(k))
    ideal = np.argsort(-gains)[:k]
    idcg = sum(gains[ideal[i]] / np.log2(i+2) for i in range(k))
    return dcg/idcg if idcg>0 else 0.0
def mrr(scores, gains):
    for i,a in enumerate(np.argsort(-scores)):
        if gains[a]>0: return 1.0/(i+1)
    return 0.0
def prec_at_k(scores, gains, k=3):

```

```

        return float(np.mean([gains[a]>0 for a in np.argsort(-scores)[:k]]))
def recall_at_k(scores, gains, k=5):
    total_pos = max(int((gains>0).sum()),1)
    return float(np.sum([gains[a]>0 for a in np.argsort(-scores)[:k]])/total_pos)
def regret(scores, gains):
    chosen = int(np.argmax(scores))
    best = float(gains.max()); chosen_v = float(gains[chosen])
    return (best - chosen_v) / (best+1e-9)

def task_gains(ti):
    skills_i = task_required_skills[ti]
    g = true_skill[:, skills_i].mean(axis=1)
    return (g > np.median(g)).astype(float)    # binary relevance

# %% [code] Cell 8 – Main benchmark over test tasks
methods = {
    "Random":      lambda ti: baseline_random(ti),
    "Perf-only":   lambda ti: baseline_perf_only(ti),
    "Sim-only":    lambda ti: baseline_sim_only(ti),
    "VanillaPR":   lambda ti: baseline_vanilla_pr(ti),
    "WeightedAvg": lambda ti: baseline_weighted_avg(ti),
    "HARP":        lambda ti: harp_rank(ti)[0],
}
results = {m: {k:[] for k in ["NDCG@5", "MRR", "P@3", "R@5", "Regret"]} for m in methods}
for ti in test_idx:
    g = task_gains(ti)
    for m,fn in methods.items():
        s = fn(ti)
        results[m]["NDCG@5"].append(ndcg_at_k(s, g, k=5))
        results[m]["MRR"].append(mrr(s, g))
        results[m]["P@3"].append(prec_at_k(s, g, k=3))
        results[m]["R@5"].append(recall_at_k(s, g, k=5))
        results[m]["Regret"].append(regret(s, g))
summary = pd.DataFrame({m:{k:np.mean(v) for k,v in d.items()} for m,d in results.items()})
print("=== Main benchmark (mean over test tasks) ===")
print(summary.round(3))

# %% [code] Cell 9 – Ablation study
abl_configs = {
    "HARP-full":      (0.5, 0.3, 0.2),
    "HARP-no-Perf":   (0.0, 0.5, 0.5),
    "HARP-no-Endor":  (0.6, 0.0, 0.4),
    "HARP-no-Sem":    (0.6, 0.4, 0.0),
    "HARP-uniform-teleport": "uniform"
}
abl_rows = []
for name,cfg in abl_configs.items():
    ndcgs = []
    for ti in test_idx:
        g = task_gains(ti)
        if cfg=="uniform":
            M_P, M_C, M_phi, A_idx, *_ = make_M_for_task(ti)
            M = 0.5*M_P + 0.3*M_C + 0.2*M_phi
            V = M.shape[0]; p = np.ones(V)/V
            r = p.copy()
            for _ in range(200):

```

```

        r = 0.85*(M@r) + 0.15*p
        s = r[A_idx]
    else:
        s,_ = harp_rank(ti, beta=cfg)
        ndcgs.append(ndcg_at_k(s, g, k=5))
        abl_rows.append((name, np.mean(ndcgs)))
abl_df = pd.DataFrame(abl_rows, columns=["Variant", "NDCG@5"]).round(3)
print("\n=== Ablation (NDCG@5) ===\n", abl_df)

# %% [code] Cell 10 – Example: show top-3 agents for example tasks
print("\n=== Example rankings (HARP) ===")
for ti in test_idx[:3]:
    s,_ = harp_rank(ti)
    order = np.argsort(-s)
    print(f"\nTask: {TASKS.iloc[ti]['desc'][:90]}...")
    for rk,a in enumerate(order[:3]):
        print(f"    #{rk+1}: {AGENT_NAMES[a]:35s}    score={s[a]:.4f}")

# %% [code] Cell 11 – Convergence plot (verifies contraction rate alpha)
ti = test_idx[0]; M_P,M_C,M_phi,A_idx,*_ = make_M_for_task(ti)
M = 0.5*M_P + 0.3*M_C + 0.2*M_phi
p = teleport_vector(ti); r = p.copy(); diffs=[]
for _ in range(60):
    r_new = 0.85*(M@r) + 0.15*p
    diffs.append(np.linalg.norm(r_new-r,1)); r = r_new
plt.figure(figsize=(6,3.5))
plt.semilogy(diffs, label="||r_{t+1}-r_t||_1")
plt.semilogy([diffs[0]*0.85**i for i in range(len(diffs))], "--", label="α^t bound")
plt.xlabel("iteration"); plt.ylabel("residual (log)"); plt.legend()
plt.title("HARP contraction – Theorem 1 empirical check"); plt.grid(True)
plt.show()

```

The notebook prints (i) the main benchmark table, (ii) the ablation table, (iii) example top-3 rankings, and (iv) a log-scale convergence plot whose slope visibly matches the theoretical α^t bound — a direct empirical check of Theorem 1.

Key Findings

- **Agent ranking is structurally analogous to early-Web ranking:** graphs of agents (AutoGen, MetaGPT, CAMEL, ChatDev, GPTSwarm) provide link structure; benchmark traces (HAL Holistic Agent Leaderboard for GAIA, AgentBench, SWE-bench) provide the performance signal; and sentence embeddings provide topical similarity.
- **Existing routers collapse this to a supervised point estimate and discard the graph.** RouteLLM (Ong et al., 2024) classifies queries as weak/strong; FrugalGPT (Chen, Zaharia, Zou, 2023) cascades to match GPT-4 with 98% cost reduction; dueling-bandit routers (arXiv:2510.00841) compare two LLMs at a time. None of these uses agent-agent edges. Existing reputation systems (EigenTrust, TrustRank) ignore semantics and task-conditioning. [\(arXiv\)](#)
- ****HARP is the smallest formulation that retains PageRank's mathematical guarantees while genuinely fusing three signals.**** By combining three column-stochastic operators *convexly*, the composite remains column-stochastic and Perron-Frobenius still gives a unique stationary score; Theorem 1 gives a contraction rate of exactly α in ℓ_1 .

- **The task-conditional softmax teleport generalizes Haveliwala's 16-topic teleport** to a continuous embedding space, which is critical for cold-start agents (Theorem 4): with no history, HARP reduces gracefully to semantic-only ranking — the same formal limit as Topic-Sensitive PageRank.
- **Benchmark selection matters.** HAL (Kapoor, Stroebel et al., arXiv:2510.11977, ICLR 2026) currently exposes 32 evaluations on the GAIA 165-question public-validation split with downloadable per-task trace data, making it the only benchmark today where the (agent, task) success matrix is dense and public — exactly what HARP needs for $\mathbf{M}^P$. [\(princeton\)](#)

Details

(See Parts I, II, and III above for the full derivation, math, and code.)

Recommendations

1. ****For production agent routers**:** deploy HARP as the **prior** for a Thompson-sampling exploration policy. Use $(\beta_P, \beta_C, \beta_\phi) = (0.5, 0.3, 0.2)$ and $\alpha = 0.85$ as defaults; tune κ in $[5, 20]$ on a held-out task slice. Recompute the stationary vector lazily per incoming task (~ 85 sparse iterations for $\varepsilon = 10^{-6}$).
2. **Switch to a richer benchmark when available:** if/when AgentBench, ToolBench, and SWE-bench release per-task $\times$ per-agent outcome matrices (HAL has done so for GAIA at huggingface.co/datasets/agent-evals/hal_traces), retrain $\mathbf{M}^P$ on the union.
3. **Add a TrustRank-style seed set** for high-stakes deployments: vet a small subset of agents, set $\mathbf{p}_\tau$ to mix the semantic component with a seed-trust indicator; this is a one-line change in [\(teleport_vector\(...\)\)](#).
4. **Re-evaluate quarterly:** rankings shift fast. SWE-bench Verified rose from 49.0% (Claude 3.5 Sonnet, October 2024) to 93.9% (Claude Mythos Preview, April 2026) — roughly 18 months. Freeze $\mathbf{M}_\tau$ at most monthly in production. [\(Substack\)](#) [\(Verdent AI\)](#)
5. **Bench thresholds that should change the recommendation:**
 - If $|H| < 5$ outcomes per (agent, skill), set $\beta_P \leq 0.2$ — the Beta posterior is too flat.
 - If endorsement edges are scraped from logs rather than curated, set $\beta_C \leq 0.15$ and add adversarial-edge filtering (Spam Mass, Gyöngyi et al., 2006).
 - If the embedding model is replaced by a stronger reranker (BGE-M3, GTE-large), raise β_ϕ to 0.3-0.4.
 - If you observe NDCG@5 gain over Sim-only drop below 3 percentage points across 200+ tasks, H is too sparse — fall back to Topic-Sensitive PageRank.

Caveats

- HAL is currently the only public source of dense per-task $\times$ per-agent outcomes for GAIA; for other benchmarks, the matrix H must be reconstructed from individual paper appendices.
- The "endorsement" signal is partly synthetic in our experiments: real agent-to-agent invocation logs are not yet centralized. Co-success is a reasonable proxy but is correlated with the performance signal — interpret β_C ablations with care.
- Tensor-PageRank / Multilinear PageRank extensions sacrifice uniqueness for higher-order structure (Gleich, Lim, Yu, SIAM J. Matrix Anal. Appl., 2015, which explicitly notes that *"there are parameter regimes of multilinear PageRank where solutions are not unique"*); we deliberately stay in the matrix regime. [\(arxiv\)](#)
- Reported SWE-bench scaffold numbers are vendor-self-reported and have been shown to overstate true correctness: SWE-ABS (arXiv:2603.00520, March 2026) reports that

adversarial-strengthened tests reject 19.78% of previously passing patches, dropping the top agent's score from 78.80% to 62.20% and reshuffling the top-30 leaderboard. Apply equivalent skepticism to any benchmark that feeds $\mathbf{M}^P$. [\(arxiv\)](#)

- Ethical: high-rank entrenchment (Matthew effect) is real. Always pair HARP with an exploration bonus or audit log; the seed-trust extension in §7 is the natural starting point.